

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL 2- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	SHAIK HAZIPEERA	Reg.No.:	192572056
Department:	CSE-AI	Date:	25/07/2026

ANNEXURE A

1. A Booth's multiplication program produces an incorrect product for the inputs -13×9 . Debug the algorithm by identifying the incorrect shift or addition/subtraction step and optimize the implementation to reduce execution cycles.
2. A processor implementing the **restoring division algorithm** generates an incorrect quotient for $29 \div 5$. Debug the iteration sequence, identify the faulty restore operation, and propose an optimized correction.
3. During the execution of the **non-restoring division algorithm**, the quotient bits are incorrect after the fourth iteration. Identify the logic error, correct the algorithm, and verify the final quotient and remainder.
4. A Booth multiplier performs arithmetic right shifts as logical shifts, resulting in an incorrect signed multiplication result. Debug the issue and explain how arithmetic shifting improves correctness.
5. A floating-point addition program based on the **IEEE 754 single-precision standard** gives an incorrect result because the exponents are not aligned properly. Identify the bug and optimize the alignment process

Code of Conduct:

I SHAIK HAZIPEERA (192572056) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

S. Hazipeera

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

ANSWER:

1. Booth's Multiplication (-13×9)

Given

- Multiplicand (M) = -13
- Multiplier (Q) = 9

Expected Result:

$$-13 \times 9 = -117 \quad -13 \times 9 = -117$$

Error Identified

The program performs the **wrong addition/subtraction before the arithmetic right shift**.

Correct Booth rules:

- **01** $\rightarrow A = A + M$
- **10** $\rightarrow A = A - M$
- **00 or 11** $\rightarrow$ No operation
- Then perform **Arithmetic Right Shift (ARS)**.

If subtraction is performed instead of addition (or vice versa), the final product becomes incorrect.

Correction

- Execute the proper add/subtract operation according to the Booth bit pair.
- Always perform **Arithmetic Right Shift**, not logical shift.

Optimization

- Skip cycles when the bit pair is **00 or 11** (only shift required).
- Merge add/subtract hardware using a single adder.

Final Product

-117

2. Restoring Division ($29 \div 5$)

Given

Dividend = 29

Divisor = 5

Expected:

$29 \div 5 = 5, \text{Remainder} = 4$

$\text{Remainder} = 4$

Error Identified

After subtraction,

If

$A - M < 0$

the processor **did not restore** the accumulator.

Correct restoring step:

If A becomes negative,

$A = A + M$

and quotient bit = 0.

Correction

Restore the accumulator immediately after every negative subtraction.

Optimization

- Detect negative result quickly.
- Skip unnecessary restore logic using improved control circuitry.

Correct Result

Quotient:

5\boxed{5}5

Remainder:

4\boxed{4}4

3. Non-Restoring Division

Error

After the **4th iteration**, the quotient bit was generated incorrectly.

Correct rule:

- If accumulator is positive $\rightarrow$ subtract divisor $\rightarrow$ quotient bit = 1
- If accumulator is negative $\rightarrow$ add divisor $\rightarrow$ quotient bit = 0

The implementation assigned the opposite quotient bit.

Correction

Generate quotient bits according to the accumulator sign after each operation.

If the final accumulator is negative,

perform one final restore:

$$A=A+MA=A+MA=A+M$$

Verified Result ($29 \div 5$)

Quotient:

$$5\boxed{5}5$$

Remainder:

$$4\boxed{4}4$$

4. Booth Multiplier Using Logical Shift Instead of Arithmetic Shift

Problem

The multiplier performs a **Logical Right Shift**.

Logical shift inserts:

0

into the MSB.

For signed numbers, this changes the sign bit and produces an incorrect product.

Example:

11110010

Logical shift $\rightarrow$

01111001

Sign becomes positive (incorrect).

Correct Operation

Arithmetic Right Shift copies the sign bit.

Example:

11110010

Arithmetic shift →

11111001

Sign remains negative.

Why Arithmetic Shift?

- Preserves two's complement sign.
- Maintains signed value during every Booth iteration.
- Produces the correct signed multiplication result.

5. IEEE 754 Floating-Point Addition

Problem

Two floating-point numbers have different exponents.

The program adds mantissas directly without aligning exponents.

Example:

1.101×2^5

+

1.011×2^3

Direct addition is incorrect.

Correct Procedure

1. Compare exponents.
2. Shift the mantissa of the smaller exponent to the right until exponents match.
3. Add mantissas.
4. Normalize the result.
5. Round according to IEEE 754.
6. Store sign, exponent, and mantissa.

Bug

Mantissas were added before exponent alignment.

Optimization

- Use a barrel shifter for fast alignment.
- Skip alignment if exponents are already equal.
- Perform normalization only when necessary.

Correct Result

Proper exponent alignment ensures an IEEE 754-compliant floating-point sum.

Summary Table

Question	Error	Correction	Final Result
1. Booth Multiplication	Wrong add/subtract or shift	Follow Booth rules and use Arithmetic Right Shift	-117

Question	Error	Correction	Final Result
2. Restoring Division	Restore step omitted	Restore accumulator when subtraction is negative	Quotient = 5, Remainder = 4
3. Non-Restoring Division	Wrong quotient bit after 4th iteration	Set quotient from accumulator sign and perform final restore if needed	Quotient = 5, Remainder = 4
4. Booth Shift Error	Logical shift used	Replace with Arithmetic Right Shift	Correct signed multiplication
5. IEEE 754 Addition	Exponents not aligned	Align exponents before mantissa addition, then normalize and round	Correct floating-point result